



ADS02: 红黑树与 B+ 树

来源: Materials/Student Slides/ADS02BTree_Stu.ppt
课件范围: S1-S3、S4-S8、S9-S12、S13; 共 13 张幻灯片。
笔记按原课件顺序整理; S 编号指幻灯片顺序。

本讲路线图

- 红黑树的不变量与高度界 → 插入、删除的局部修复 → B+ 树的节点容量与操作 → 高度和时间代价。

1. 红黑树的结构与高度保证 [S1-S3]

红黑树 (Red-Black Tree, RB Tree) 通过颜色限制根到叶的路径长度, 是一种保持对数高度的二叉搜索树 (Binary Search Tree, BST)。[S1-S2]

1.1 五条红黑性质 [S2]

重要性: A · 必须掌握

每个节点除键、左右孩子和父节点外, 还带一个颜色位。空孩子用黑色哨兵叶节点 NIL 表示。[S2]

1. 每个节点是红色或黑色。
2. 根为黑色。
3. 每个 NIL 叶节点为黑色。
4. 红节点的两个孩子均为黑色, 即不存在连续的红节点。
5. 从任意节点到其后代 NIL 叶节点的每条简单路径, 黑节点数相同。

前四条限制局部颜色, 第五条把不同分支的“黑色骨架”长度锁定在同一数量; 两者共同限制整棵树的高度。[S2]

1.2 黑高与对数高度证明 [S3]

重要性: A · 必须掌握

节点 x 的黑高 (Black Height) $bh(x)$ 是从 x 之后到任意后代 NIL 叶节点路径上的黑节点数; 不计 x 自己。红黑性质 5 保证它与所选路径无关。[S3]

核心原理

先用归纳法证明黑高为 b 的子树至少有 $2^b - 1$ 个内部节点, 再用“红节点不能相邻”将普通高度与黑高联系起来。

要证明高度 $h = O(\log N)$, 关键是黑高每增加一层, 所需最少节点数近似翻倍。设 $n(x)$ 为以 x 为根的内部节点数:

$$n(x) \geq 2^{bh(x)} - 1.$$

归纳基：NIL 的黑高为 0，内部节点数为 $0 = 2^0 - 1$ 。归纳步： x 的每个孩子 c 的黑高至少为 $bh(x) - 1$ ；若 x 是黑色，孩子少一层黑高，若 x 是红色，孩子黑高与 x 相同。因此两个孩子各至少有 $2^{bh(x)-1} - 1$ 个内部节点，得到

$$n(x) \geq 1 + 2(2^{bh(x)-1} - 1) = 2^{bh(x)} - 1.$$

任一路径上不能有相邻红节点，所以普通高度至多是黑高的两倍： $h \leq 2bh(\text{root})$ 。又因为 $N \geq 2^{bh(\text{root})} - 1$ ，故

$$h \leq 2\log_2(N + 1) = O(\log N).$$

补充推导：课件把“完成证明”留作讨论。上式用 $bh(\text{root}) \leq \log_2(N + 1)$ 与 $h \leq 2bh(\text{root})$ 相乘得到，并把归纳步中两个孩子的下界明确写出。[S3]

记号核对：S3 的图面写作 $2\ln(N + 1)$ ，但归纳式使用 2^{bh} ，严格推出的上界是 $2\log_2(N + 1)$ 。若 $\ln$ 按自然对数理解，缺少因子 $1/\ln 2$ ；见“待核对”。

2. 红黑树的更新 [S4–S8]

搜索沿树高进行；插入和删除先按 BST 的顺序关系修改局部，再用颜色变更与旋转恢复红黑性质。[S4–S8]

2.1 插入的三种局部情形 [S4]

重要性：A · 必须掌握

新键先像普通 BST 一样插入，并把新节点染红。红色不会改变经过它的路径黑节点数；若它的父节点也是红色，才违反“红节点不能相邻”。令新节点为 z ，其父为 p ，祖父为 g ，叔父为 u 。[S4]

核心原理

只处理新节点与红父节点造成的局部冲突：叔父红时换色并向上检查，叔父黑时通过旋转把折线变直，再重新着色。

1. **情形 1：叔父 u 为红色。**把 p, u 染黑， g 染红；经过 g 的各路径黑高不变，但若 g 与其父同为红色，继续向上处理。
2. **情形 2：叔父黑且 z, p, g 呈“折线”。**先绕 p 旋转，将结构化为情形 3 的“直线”。
3. **情形 3：叔父黑且三者呈“直线”。**绕 g 旋转，并交换新局部根与旧祖父的颜色，使红红冲突消失且黑高不变。

上述以左侧冲突为例；右侧完全镜像。最后把整棵树根染黑。课件图中的连续插入示例展示了修复可以逐级传递，时间为 $O(h) = O(\log N)$ ，一次插入最多两次旋转。[S4、S8]

解释补充：旋转改变父子关系，但保持相关子树的中序先后顺序，因此不破坏 BST 键序。染色是为保持路径黑节点数与消除相邻红节点。[S4]

2.2 删除的基本步骤与“缺一层黑色” [S5]

重要性：A · 必须掌握

先按 BST 规则删除：无孩子的节点直接摘除；仅有一个孩子的节点用孩子替代；有两个孩子的节点用左子树最大键或右子树最小键替换，再删除该替换节点。若实际被摘除的位置原来是黑色，经过该位置的路径会少一层黑色，需要修复；红节点被摘除时不会减少黑高。[S5]

核心原理

把删除黑节点造成的路径黑高不足视为节点 x 的“额外黑色”，再根据兄弟 w 与其孩子的颜色选择换色、旋转或向上传递。

修复目标是在不破坏 BST 顺序的同时恢复所有根到 NIL 路径的共同黑高。若黑节点只有一个红孩子，替代后可直接把孩子染黑；一般情形按下节处理。[S5]

2.3 删除修复的四种情形 [S5–S7]

重要性：A · 必须掌握

以下以 x 是父节点的左孩子、兄弟 w 在右侧为例，右侧缺黑时完全镜像。S5–S6 的图给出四种颜色结构，S7 用具体键值串联多次调整。[S5–S7]

1. **情形 1：兄弟 w 为红色。** 父节点必为黑色。交换父与兄弟的颜色，绕父节点左旋；此时 x 有一个黑色兄弟，转入情形 2、3 或 4。旋转本身尚未消除缺黑。
2. **情形 2：兄弟黑色且两个孩子均黑色。** 把 w 染红，让兄弟侧也少一层黑色；若父节点原为红色，把父染黑即可结束，否则将缺黑位置上移到父节点，继续处理。
3. **情形 3：兄弟黑色、近侧孩子红、远侧孩子黑。** 将近侧孩子染黑、 w 染红，绕 w 右旋，转化为情形 4。
4. **情形 4：兄弟黑色、远侧孩子红。** 让 w 取得父节点原颜色，父与远侧孩子染黑，再绕父节点左旋；路径黑高恢复，修复结束。

S7 的示例说明情形 1、2 可以引出后续情形；不应把每张图当成四个相互独立的删除算法。整次删除沿高度 $O(\log N)$ 调整，旋转次数至多 3；作为比较，课件列出 AVL 删除可能需要 $O(\log N)$ 次旋转。[S7–S8]

3. B+ 树的结构与操作 [S9–S12]

B+ 树 (B+ Tree) 用多路分支降低树高，内部节点主要导航，实际记录存于叶节点。[S9]

3.1 阶数、容量与叶层 [S9]

重要性：A · 必须掌握

按课件的 M 阶约定，根节点要么就是叶节点，要么有 2 到 M 个孩子；非根内部节点有 $\lceil M/2 \rceil$ 到 M 个孩子；所有叶节点深度相同。内部节点最多有 M 个孩子指针，使用后面 $M - 1$ 个子树的最小键作为分隔键；完整数据留在叶节点。[S9]

搜索从根按分隔键选择子树，直到叶节点取出记录。图中的 $M = 4$ 示例显示分隔键 21、48、72 对应四个连续键区间；叶节点保存实际键集合。[S9]

课件还给出非根叶节点的装载下界与上界，但原句使用“children”描述叶节点，计数单位与通常的“叶中记录数”不一致，见“待核对”。[S9]

3.2 查找、插入与删除 [S10–S12]

重要性：A · 必须掌握

S10 的 3 阶树先展示查找 52，再连续插入 18、1、19、28；S11 接着插入 70，强调优先考察可调整的兄弟节点。数字图用来显示叶节点内容与内部路由键会一起变化；操作规则应以容量约束为准。[S10–S11]

核心原理

插入时先到正确叶节点；若局部容量超限，就分裂并把新的路由信息向父节点传播，必要时新建根节点。

1. 从根搜索到应含新键的叶节点，在叶中按序插入。
2. 若节点溢出，课件伪代码用“出现 $M + 1$ 个键”表示超限，并把键分到两个节点，数量分别约为 $\lceil (M + 1)/2 \rceil$ 与 $\lfloor (M + 1)/2 \rfloor$ 。
3. 更新父节点的分隔键与孩子指针。父节点若也溢出，继续向上分裂；原根分裂时建立含两个孩子的新根。[S12]

解释补充：叶节点分裂时完整记录仍在叶层，内部节点只维护指向孩子的路由键。课件伪代码把叶键数和内部节点的分支数压缩成同一种“ $M + 1$ keys”说法；实际实现需分别检查容量定义。[S9、S12]

删除时先从叶层移除键。若节点低于最小装载量，先向富余的相邻兄弟借键并更新父分隔键；借不到时与兄弟合并，再检查父节点是否下溢。课件强调寻找有两个键的兄弟进行调整；根若只剩一个孩子，可删除旧根，让该孩子成为新根。[S11]

3.3 高度与代价模型 [S12]

重要性：B · 重点理解

若每个非根内部节点至少有 $\lceil M/2 \rceil$ 个孩子，树高随键数 N 增长为

$$H = O\left(\left\lceil \log_{\lceil M/2 \rceil} N \right\rceil\right).$$

课件给出节点内线性处理 $O(M)$ ，因而总操作时间 $O\left(\frac{M}{\log M} \log N\right)$ ；若在节点内通过有序键做二分查找，则查找时间可写成 $O(\log N)$ 。这里比较的是内存中的键比较代价。[S12]

解释补充：课件提出“最佳 M 为 3 或 4”是上述 $M/\log M$ 代价模型的结论。在外存或数据库中，每次磁盘页访问的代价通常更关键，较大的节点容量可能更有利；选择 M 要先明确成本模型。[S12]

4. 阅读材料 [S13]

重要性：C · 了解即可

红黑树与 B 树相关章节可参考 *Introduction to Algorithms*, 3rd ed., Chapter 13 与 Chapter 18。[S13]

本讲总结

- 红黑树用根、NIL、相邻红节点和等黑高等约束，将高度控制在 $O(\log N)$ 。[S2–S3]
- 插入新节点先染红；红叔父换色，黑叔父通过旋转把折线变直并修复。[S4]
- 删除黑节点会导致局部缺黑；根据兄弟与其子节点颜色处理，最坏沿树高传播。[S5–S8]
- B+ 树的内部节点存路由键，数据在等深的叶层；溢出时分裂并向上传播。[S9–S12]
- B+ 树的性能取决于分支数、节点内搜索方法与内存或外存的成本模型。[S12]

复杂度与适用条件速查

结构 / 操作	时间复杂度	旋转或结构调整	条件	来源
红黑树搜索	$O(\log N)$	无	红黑性质保持	S2–S3
红黑树插入	$O(\log N)$	至多 2 次旋转	先按 BST 插入并染红	S4、S8
红黑树删除	$O(\log N)$	至多 3 次旋转	黑节点删除可能缺黑	S5–S8
B+ 树高度	$O(\log_{\lceil M/2 \rceil} N)$	分裂或合并可逐级传播	最小分支数约束	S9、S12
B+ 树操作	$O((M/\log M) \log N)$	每层线性处理 $O(M)$	课件的内存成本模型	S12
B+ 树查找	$O(\log N)$	节点内有序键搜索	每层二分定位	S12

易错点

- 黑高不计节点 x 自身，且 NIL 叶节点算黑色；高度和黑高是不同量。[S2–S3]
- 插入情形 2 是把折线转成直线的过渡，通常还要继续执行情形 3。[S4]
- 删除情形 1 的旋转只改变兄弟的颜色结构；缺黑尚未解决。[S5–S6]
- B+ 树内部的分隔键用于导航，完整记录在叶节点；不要把路由键当成仅存在于内部的记录。[S9]
- 课件的 $M = 3$ 或 4 结论依赖其节点内操作模型，不能直接当作磁盘 B+ 树的普遍最佳阶数。[S12]

复习自测

1. 红黑树的哪两条性质共同保证 $h = O(\log N)$ ？补全黑高归纳证明。（来源：[S2–S3]）
2. 插入后父节点和叔父都是红色时怎样恢复性质？若叔父黑色呢？（来源：[S4]）
3. 删除黑节点后，兄弟为红、兄弟与两孩子均黑、仅近侧孩子红、远侧孩子红时各如何处理？（来源：[S5–S7]）
4. 3 阶 B+ 树发生叶节点溢出时，为什么父节点也可能需要分裂？（来源：[S9–S12]）
5. 为什么 B+ 树的阶数选择不能只看 $O((M/\log M) \log N)$ ？（来源：[S12]）

术语速查

术语	概念	来源
红黑树 (Red-Black Tree, RB Tree)	用颜色不变量保持对数高度的 BST	S2
哨兵叶节点 (NIL Sentinel)	代表空孩子的黑色外部节点	S2

术语	概念	来源
黑高 (Black Height)	从节点之后到 NIL 的共同黑节点数	S3
旋转 (Rotation)	保持中序键序的局部父子关系重组	S4–S6
B+ 树 (B+ Tree)	内部节点导航、数据在叶层的多路平衡搜索树	S9
分隔键 (Separator Key)	指示子树键区间边界的内部路由键	S9
分裂 (Split)	节点溢出后分成两个节点并更新父节点	S12

待核对

- [待核对 S3] 图面写 $h \leq 2\ln(N + 1)$ ，但同页的 2^{bh} 归纳式严格导出 $h \leq 2\log_2(N + 1)$ ；建议对照任课教师采用的 $\ln$ 记号或更正信息。
- [待核对 S9] 原文说非根叶节点有 $\lceil M/2 \rceil$ 至 M 个 “children”，而叶节点装的是数据；具体实现中应核对叶节点容量按记录数还是其他单位计。
- [待核对 S10–S11] 数字动画的每一步中间树形没有可可靠重建的原生文字；如需逐键复现，可对照原幻灯片图示。